Промпт для агента сокомандника

Задача

Мы продолжаем университетский проект Loki TVA Escape на Java 21, LibGDX 1.14 и Maven. Не создавай новый проект. Пиши простой понятный код, который мы сможем объяснить на защите. Сохрани существующую историю Git и готовую графику.

Цель: Локи перемещается по лабиринту TVA, выполняет задания и выходит через портал. Четыре разных охранника ведут себя по логике, вдохновлённой Pac-Man. Это наша адаптация, а не точная копия оригинала.

Как работать

Сначала выполни только этап 1: подключение четырёх готовых охранников. Остальные этапы ниже - план следующих итераций. После первого этапа собери проект и объясни, какие файлы изменены и что проверено. Не реализуй все этапы одним большим изменением.

Проект и запуск

Корневая папка: SDP_Final. На исходном компьютере она находится по адресу C:/Users/Anyap/Downloads/SDP/SDP_Final. У тебя путь может отличаться. Запуск из корня: .\run.cmd. Проверка сборки: mvn clean package. Java 21 и Maven должны быть доступны в PATH.

pom.xml содержит зависимости и настройки запуска. Maven запускает игру с рабочей папкой assets. Оставь один Maven-проект: без Gradle, дополнительных модулей и лишней вложенности папок.

Текущее состояние

Уже есть карта со стенами, движение Локи, анимация, патруль и преследование обычного охранника, столкновения, урон, сбор предметов и портал. Движение пока свободное, включая диагонали. На карте только один G, используется guard.png.

Четыре новых охранника уже нарисованы, но ещё НЕ подключены в игру. Верхняя отладочная надпись и полоса убраны - не возвращай их. Сообщения паузы, победы и поражения остаются.

Управление: WASD/стрелки - движение; Space - рывок; E - иллюзия; R - перезапуск; Esc - пауза. Перед началом убедись, что получил актуальную папку и новые PNG: они могут ещё не быть отправлены в удалённый репозиторий.

Карта файлов и готовая графика

Вход и экран

src/loki/GameLauncher.java - main и создание окна LWJGL3.
src/loki/game/LokiGame.java - создание игрового экрана.
src/loki/game/GameScreen.java - ввод, текстуры, отрисовка и пауза. Правила игры держи отдельно.

Игровые классы

src/loki/game/CharacterAnimation.java - направление и кадры ходьбы.
src/loki/game/Player.java - положение, движение, столкновения со стенами.
src/loki/game/Guard.java - охранник, патруль, видимость, память цели и текущий поиск пути.
src/loki/game/Illusion.java - приманка Локи.

Правила и паттерны

src/loki/facade/GameSession.java - состояние игры, охранники, способности, предметы, урон, победа и проигрыш.
src/loki/builder/Level.java - клетки, карта, спавны, BFS и Level.Builder.
src/loki/strategy/ - GuardStrategy, PatrolStrategy, ChaseStrategy.
src/loki/factory/ - LokiCreator, Loki, LokiVariant.
src/loki/decorator/ - EquipmentDecorator, MagicAmplifier, TimeShield.
src/loki/observer/GameListener.java - уведомления о состоянии игры.

Изображения в assets/

loki.png - Локи; guard.png - прежний обычный охранник.
b15.png - B-15; d90.png - D-90; c20.png - C-20; x5.png - X-5.
interior.png - четыре клетки пола/стен 48×48 в одном ряду.
objects.png - открытый портал, закрытый портал, TemPad: три клетки 48×48.
README.md - описание изображений и ссылки на исходники Canva.

Как устроены атласы персонажей

Каждый PNG имеет размер 192×288. Один кадр - 64×96. Три столбца: покой, шаг A, шаг B. Ряды сверху вниз: вниз, вверх, вправо. Влево получается отражением правого ряда. Используй существующие createSprites() в GameScreen и CharacterAnimation. Не перерисовывай и не заменяй готовые изображения.

Этап 1. Четыре охранника

Выполни сейчас

1. Добавь простой GuardType с B15, D90, C20, X5. Каждый Guard хранит свой тип. Тип определяет изображение и в дальнейшем поведение.

2. Добавь четыре доступных спавна G на карту в Level.training(). Назначь каждому свой тип явно и предсказуемо: каждый из четырёх типов ровно один раз.

3. Загрузи четыре текстуры один раз в GameScreen. Нарежь их существующим способом. У каждого охранника должен быть собственный CharacterAnimation. Освободи текстуры в dispose(). Не загружай PNG внутри render().

4. При рисовании выбирай атлас по типу самого Guard. Не связывай внешний вид с порядковым номером в списке: изображение и будущая логика должны относиться к одному типу.

5. Пока сохрани существующие патрулирование и преследование. Не внедряй режимы Pac-Man на этом этапе. Оставь guard.png в assets, даже если новый состав его пока не использует.

Проверка этапа

Выполни mvn clean package. Запусти игру и проверь, что видны четыре разных охранника, каждый ходит и меняет кадры при смене направления. Проверь столкновения со стенами, паузу и перезапуск R. Если запуск недоступен, честно напиши, что проверена только сборка.

Что сообщить после работы

Коротко перечисли изменённые файлы, способ назначения типов и проведённые проверки. Не утверждай, что поведение Pac-Man уже реализовано: сейчас подключаются только персонажи.

Что сохраняем

Работающие Builder, Factory Method, Decorator, Strategy, Facade и Observer. Не добавляй пустые классы ради названий паттернов и не удаляй GameListener из-за удаления старой верхней надписи.

Этапы 2 и 3. Движение и охота

Этап 2 - движение по клеткам

Клетка Level.TILE равна 48. Персонажи плавно движутся между центрами клеток в четырёх направлениях, без диагоналей. Следующий поворот запоминается и выполняется в ближайшем допустимом центре. Если повернуть нельзя, продолжается текущее допустимое движение. Определи, как рывок работает с этой системой: он тоже не должен проходить сквозь стены.

Проверить повороты, узкие проходы, тупики, быстрое нажатие следующего направления, паузу и R. Перемещение рассчитывать по игровому времени, а не по числу кадров.

Этап 3 - режимы

Общие режимы CHASE и SCATTER. Для первой версии достаточно повторять 7 секунд SCATTER и 20 секунд CHASE. Это наши настройки. На паузе таймер стоит, при R сбрасывается. В SCATTER у каждого охранника свой угол.

Наши характеры в CHASE

B-15: текущая клетка Локи - прямое преследование.
D-90: четыре клетки впереди Локи - перехват.
C-20: точка на две клетки впереди Локи; цель = 2 × эта точка − клетка B-15 - обход с другой стороны.
X-5: преследование при расстоянии не меньше восьми клеток; вблизи - уход к своему углу. Это придуманные правила нашей игры, не характеристика персонажей сериала.

Выбор направления

На центре клетки сравнивать соседние проходы по квадрату расстояния до цели. Обычно исключать движение назад, в тупике разрешать. Равенство решать порядком UP, LEFT, DOWN, RIGHT. Между центрами направление сохраняется. Цель может быть вне карты или в стене: доступным должен быть соседний проход. При смене режима разворот можно разрешить явно.

Важно при замене старого поведения

GameSession сейчас каждый кадр вызывает setStrategy() по isSearching(). Замени этот переключатель, иначе он затрёт новую логику. CHASE не ограничивается прямой видимостью. Иллюзии можно дать явный временный приоритет. Не запускай BFS каждый кадр вместо выбора на перекрёстке.

Технический ориентир: The Pac-Man Dossier - https://pacman.holenet.info/ . Используй идеи, без обязательного воспроизведения багов и точных таймингов оригинала.

Этап 4. Задания и выход

Минимальный сценарий следующей итерации

Сначала Локи собирает три существующих TemPad (C), затем активирует один терминал рядом с ним клавишей F. После этого открывается портал. Вход в открытый портал завершает игру победой. E уже занята иллюзией.

Карта и состояние

Если вводишь символ T для терминала, обнови допустимые символы, разбор карты и проверку доступности в Level.Builder. Все задания должны быть достижимы. Для временного терминала можно использовать существующую графику с понятным отличием; отдельный рисунок согласуем позже.

Единое условие открытия

Сейчас GameScreen выбирает изображение открытого портала по charges().isEmpty(), а GameSession отдельно проверяет это для победы. Введи единое условие, например isExitOpen(), и используй его в обоих местах. Рисунок и правила должны совпадать. Все состояния заданий сбрасываются при R.

Интерфейс

Для прогресса используй существующий GameListener. Если добавляешь отображение задания, сделай его компактным и понятным. Не возвращай старую отладочную простыню сверху.

Общие ограничения

Не добавляй JUnit, папки tests и новые проверки в сборку: ранее их убрали по решению автора. Проверяй сборку и поведение на каждом этапе. Не меняй готовую графику, не создавай дополнительные менеджеры, слои и зависимости без необходимости.

Не делай commit/push без отдельной просьбы. Не переписывай историю и не создавай фиктивную совместную работу. Меняй только нужные файлы и объясняй результат простыми словами.

Первое действие агента

Осмотри актуальные файлы и выполни этап 1. Затем остановись с результатом сборки и кратким отчётом. Этапы 2–4 остаются планом дальнейшей работы.